Contrast to the Past:

Autoregressive Contrastive Temporal Knowledge Graph Extrapolation

Maximilian von Bonsdorff
maxvb@kth.se

Alexander Gutell
agutell@kth.se

Sixten Norelius
sixtenn@kth.se

Emilia Sjögren
emsjog@kth.se

***Abstract*—Graphs are fundamental across various fields, representing complex relationships and structures—such as social networks, chemical bonds, and even neural networks in machine learning. Knowledge graphs extend this concept to organize real-world information into entities and relationships, structuring facts in interconnected ways. However, traditional knowledge graphs lack temporal context, limiting their ability to capture the evolution of information over time. Temporal Knowledge Graphs (TKGs) address this by adding a temporal dimension, enabling dynamic analysis and prediction of future events by tracking how interactions evolve across timestamps.**

This project seeks to improve the RE-GCN model for extrapolation tasks on TKGs, specifically for predicting events involving unseen entities or relationships. By integrating a contrastive learning framework inspired by the CENET model, our approach enhances RE-GCN's ability to distinguish historical from non-historical dependencies, which in turn improves predictive accuracy on complex, dynamic datasets. Experimental results on the ICEWS18 dataset demonstrate that our modifications to RE-GCN lead to unchanged accuracy in capturing both historical and non-historical dependencies. These results underscore the complexity of modeling diverse temporal patterns within TKGs and highlight areas for further refinement, such as improved calibration techniques and extensions for uncertainty quantification.

I. Introduction

Graphs are all around us. One of the most common examples of graph representation is social networks, where each node represents a person, and the edges represent the relationships between them. Beyond social networks, graphs are found in many other fields—such as in the molecular structures of chemistry, where atoms are nodes and bonds between them are edges. Similarly, in machine learning, neural networks can also be understood as graphs, with neurons acting as nodes and the connections between them as edges [1].

However, graphs aren't just limited to representing physical or abstract structures like molecules or networks—they can also play a crucial role in organizing knowledge. This is where knowledge graphs come into play. Knowledge graphs represent real-world information in a structured manner through connected facts. Each fact consists of two entities and the relationship between them. For example, in a knowledge graph about famous inventors, the nodes might represent individuals like "Nikola Tesla" or "Thomas Edison," and the edges could show the relationships between them, such as "collaborated with," "competed against," or "invented."

Traditional knowledge graphs lack temporal information, limiting their ability to capture the evolution of facts over time. Temporal knowledge graphs (TKGs) address this by incorporating time into each fact, creating quadruples of information: (subject, relation, object, time) [2]. This temporal dimension provides a dynamic view of knowledge, enabling us to track how information evolves, Figure 1. Predicting future facts in a TKG requires models to delve deeply into historical facts, where entities influence each other through concurrent facts at each timestamp, forming knowledge graphs (KGs) with complex structural dependencies [3].

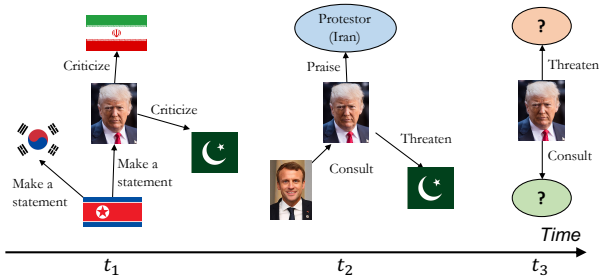


Fig. 1. Example of temporal knowledge subgraphs evolving over time. The edges represents an interaction between entities and is associated with temporal information. Our objective is to predict future interactions and construct graphs for subsequent time intervals [4].

Reasoning over a TKG can involve so called *interpolation* or *extrapolation*. Interpolation infers missing facts within observed timestamps, while extrapolation predicts future events beyond observed timestamps, a more challenging task [3]. Extrapolation is essential for understanding hidden factors in events and is valuable in practical applications like disaster relief and financial analysis. Temporal reasoning in this context typically includes two main subtasks: (1) Entity Prediction, identifying which entity will engage in a specified relation with another at a future time, and (2) Relation Prediction, forecasting the type of relation between two entities at a future timestamp.

This project explores ways to modify and improve the RE-GCN architecture proposed by Li et al. for extrapolation tasks on temporal knowledge graphs (TKG). RE-GCN models a TKG as a sequence of knowledge graphs and encodes historical facts into entity and relation representations to predict future entities and relations. It does so through evolution units, which use a relation-aware graph convolution network (GCN) to capture interactions within each KG at a given timestamp. Sequential patterns across timestamps are then learned through recurrent components, while static entity properties are integrated through a static-graph constraint. This

setup enables both entity and relation predictions at future timestamps based on the learned evolutional representations [3].

However, RE-GCN can struggle with non-historical dependencies [3] — connections that aren't directly rooted in past events involving the entities in a query. To address this, we propose to extend RE-GCN with contrastive learning, for better entity predictions, a technique that could help the model differentiate between relevant historical and non-historical events. Introducing contrastive learning by drawing inspiration from CENET [5], by Xu et al., we aim to improve the model's ability to generalize beyond simple historical patterns, potentially enhancing performance on challenging predictive tasks where diverse temporal dependencies are at play.

II. Related Work

There are numerous architectures and methods used for the extrapolation task in TKGs.

A. Graph Neural Networks

RE-NET (Reccurent Event Network) [4] is an autoregressive architecture based on the idea that future event entities and relations can be predicted as a sequential inference problem by learning from historical patterns. The architecture is built on a Recurrent Neural Network (RNN) *recurrent event encoder* which captures the historical context of both *global* representations from the entire graph up until the inference timestamp and *local* representations that capture information on entities and relations. These representations are computed using a Graph Convolutional Network (RGCN) [6].

In the same vein is RE-GCN [3]. The main difference is the static graph constraint that provides static information about entities. It also uses a GRU component for long-short term memory.

xERTE [7] builds a subgraph populated with the neighbors of the entity and timestamp of a particular query. Then it uses an attention-based mechanism to prioritize the relations in the subgraph with most significance.

B. Reinforcement Learning

Timetraveler [8] uses Reinforcement Learning, given a query employs an agent to find a path that leads to the correct target entity by starting at the source entity. It can move through relations at a current timestamp or move in time (temporal edges between graph snapshots). Over time, through a time-based reward policy network, the agent learns to minimize time steps and incorrect answers.

C. LLM-based

There are two types of LLM-based approaches.

1) In-context learning: For a specific query, relevant history is retrieved from the knowledge graph which includes historical triples where the entity has appeared. Then this data is organized into prompt that lets the LLM provide a prediction. ICLTKG [9] and zrLLM [10] use this method. The accuracy is generally close to other non-LLM approaches, but does not outperform them.

2) Supervised fine-tuning: In addition to the previous methodology, parameter-efficient fine-tuning is used to make the LLM incorporate more of the logic of temporal event sequences. Models using these technique are Chain of History [11] and GenTKG [12].

D. Other Approaches

Although a conceptually similar model to Re-NET and RE-GCN but lacks the RGCN aspect is TiRGN [13] that differentiates itself by modeling a more complete representation of historical information such as periodicity. Which according to [14] is the state of the art model (of the models with published code) on the ICEWS18 dataset and highly performant on the other common benchmark datasets. Another approach is CENET [5] which uses contrastive learning to learn to distinguish historical from non-historical queries. A classifier is then trained on these contrastive embeddings to generate a mask promoting the appropriate entities. Models such as L2TKG [15] try to learn latent relations between entities that might not be explicitly linked in the data but still have an impact on future events.

E. Benchmarks

As the extrapolation problem and the models that tackle this are relatively new, a common benchmark and methods to compare these models hasn't been formally established [16]. Although most models are in theory evaluated on the same datasets, it has been found that some of the datasets even though they bear the same name, have different data. In addition, some papers compare their *one-step* prediction models to *multi-step* prediction statistics. Furthermore, some models are evaluated on different filters such as *static*, *time-aware*, *raw*, which obviously leads to many different results for the same model depending on which paper one is reading. However, a more unified benchmarking test has been made [14] which gives a clearer picture of state-of-the-art. Finally, by introducing some simple benchmark models that obey naive rules such as counting historical occurrences of triples (or parts of triples) to predict entities, the authors are able to establish a baseline that shows a more realistic interpretation of results for all models.

III. Background

A. Notations

We collect some of the main notations from Section III-B in Table I.

B. Preliminaries

In line with the notations from the RE-GCN paper [3], a temporal knowledge graph is represented as a sequence of knowledge graphs with timestamps: $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_t, \ldots\}$. Each KG $\mathcal{G}_t$ at timestamp t is defined as $\mathcal{G}_t = (\mathcal{V}, \mathcal{R}, \mathcal{E}_t)$, where $\mathcal{V}$ is the set of entities, $\mathcal{R}$ is the set of relations, and $\mathcal{E}_t$ is the set of facts true at time t. Each fact in $\mathcal{E}_t$ is described by a quadruple (s, r, o, t), where $s, o \in \mathcal{V}$, $r \in \mathcal{R}$, and t indicates the timestamp of this relation between s (subject) and o

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Description
$\mathcal{G}, \mathcal{G}^s, \mathcal{G}_t$	TKG, static graph, KG at timestamp t in the TKG
$\mathcal{V}, \mathcal{R}, \mathcal{E}_t$	entity, relation, and fact set
$\mathcal{V}^s, \mathcal{R}^s, \mathcal{E}^s$	entity, relation, and fact set in the static graph
(s, r, o, t)	quadruple/event
$\mathbf{V}_t$, $\mathbf{W}_t$	Evolutional embedding mtrx of entities and relations at t
$\mathbf{s}$, $\mathbf{r}$ and $\mathbf{o}$	Embedded vectors of s, r, and o
$\mathcal{D}_t^{s,r}$, $\mathcal{H}_t^{s,r}$	Set of historical events and entities
q	Query - for example $(s, r, ?, t+1)$

(object). For each fact (s, r, o, t), the inverse fact (o, r^{-1}, s, t) is also included in the dataset. RE-GCN uses a static graph in their model that contains unchanging facts and is noted as $\mathcal{G}^s = (\mathcal{V}^s, \mathcal{R}^s, \mathcal{E}^s)$, where $\mathcal{V}^s$, $\mathcal{R}^s$, and $\mathcal{E}^s$ are the static entity, relation, and fact sets, respectively.

It is assumed that predictions for timestamp $t+1$ depend on the knowledge graphs from the previous m timestamps. This historical KG sequence is encoded in embedding matrices for entities $\mathbf{V}_t \in \mathbb{R}^{|\mathcal{V}| \times d}$ and for relations $\mathbf{W}_t \in \mathbb{R}^{|\mathcal{R}| \times d}$, where d is the embedding dimension. Following the CENET notation, let the bold symbols $\mathbf{s}$, $\mathbf{r}$, and $\mathbf{o}$ represent the embedding vectors for s, r, and o, respectively.

To perform entity prediction, we define the following *queries*; *object prediction*: $(s, r, ?, t+1)$ and *subject prediction*: $(?, r, o, t+1)$. For the query $(s, r, ?, t+1)$, the RE-GCN model estimates the conditional probability vector of all object entities associated with the subject entity s, the relation r, and the historical knowledge graph sequence $\mathcal{G}_{t-m+1:t}$, expressed as

$$p(o|s, r, \mathcal{G}_{t-m+1:t}) = p(o|s, r, \mathbf{V}_t, \mathbf{W}_t). \tag{1}$$

With this formulations, we can predict entities based on the historical context provided by the KGs.

To define historical patterns, we introduce the following equations: *Historical events* $\mathcal{D}_t^{s,r}$ — all past events involving s and r, given as

$$\mathcal{D}_t^{s,r} = \bigcup_{k<t} \{(s, r, o, k) \in \mathcal{G}_k\}. \tag{2}$$

Historical entities $\mathcal{H}_t^{s,r}$ — the objects o involved in historical events $\mathcal{D}_t^{s,r}$ as

$$\mathcal{H}_t^{s,r} = \{o \mid (s, r, o, k) \in \mathcal{D}_t^{s,r}\}. \tag{3}$$

Entities outside $\mathcal{H}_t^{s,r}$ are termed *non-historical entities*, and *non-historical events* are represented as the set $\{(s, r, o', k) \mid o' \notin \mathcal{H}_t^{s,r}, k < t\}$. A new event is defined as any event (s, r, o, t) not found in $\mathcal{D}_t^{s,r}$.

C. RE-GCN

The RE-GCN model captures the structural dependencies within a knowledge graph at each time step, as well as the relevant sequential patterns among temporally adjacent facts and the static properties of entities. This information is integrated into the evolutional representations of entities and relations. Using the learned representations, RE-GCN enables temporal reasoning for future timestamps through various scoring functions. The model consists of two main components: an evolution unit and multi-task scoring functions, see architecture in Section A, Figure 3. The evolution unit encodes the historical KG sequence to produce evolutional embeddings for entities and relations. The multi-task scoring functions utilize these representations at the final timestamp to perform different prediction tasks.

1) Score Function — Outputs: RE-GCN utilizes a score function at the end of the network to model the conditional probability, which represents the probability score for candidate triples s, r, o. They utilize the embedding matrices for entities and relations respectively multiplied with the results from a decoder on the query entity and relation. Finally, a sigmoid function is used to translate the results into probability scores. A more detailed explanation of the whole network as well as the score function, can be found in [3].

2) Experimental results: The RE-GCN authors categorize historical and non-historical labels as *seen* and *unseen* queries, respectively. In their case study, the data is split into these two categories, demonstrating that the model performs better when two entities have interacted in the past (historical). Although the model can still make predictions when the entities have not interacted before (non-historical), the performance is significantly lower in such cases.

D. Contrastive learning

Contrastive Learning is a deep learning method for representation learning. Its objective is to map data into a representation space where similar instances are positioned close to each other, while dissimilar instances are placed farther apart. In self-supervised contrastive learning, examples are augmentedly derived from a minibatch of N examples, resulting in 2N examples i, j, which gives a loss [17]

$$\mathcal{L}_{i,j} = -\log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j)}{\sum_{k=1, k\neq i}^{2N} \exp\left(\mathbf{z}_i \cdot \mathbf{z}_k / \tau\right)}, \tag{4}$$

where z denotes the projection embeddings of sample i and $\tau \in \mathbb{R}^+$ refers to a temperature.

E. CENET

Drawing inspiration from contrastive loss, Section III-D, the CENET model utilizes this technique to capture both historical and non-historical dependencies. The model addresses the challenge in TKGs where new events may lack historical references by considering both historical and non-historical entities, see model architecture in Section B, Figure 4. CENET extrapolates and can predict entities but is not designed for relation prediction.

1) Historical and non-historical dependency: During the data pre-processing stage, the frequencies of historical entities of the given queries, $q = (s, r, ?, t)$ are calculated. The frequencies $\mathbf{F}_t^{s,r} \in \mathbb{R}^{|\mathcal{E}|}$ of all objects associated with subject s and relation r before time step t, which yields

$$\mathbf{F}_t^{s,r}(o) = \sum_{k<t} |\{o|(s, r, o, k) \in \mathcal{G}_k\}|. \tag{5}$$

The frequency vector is then transformed into

$$\mathbf{Z}_t^{s,r}(o) = \lambda \cdot (\Phi\{\mathbf{F}_t^{s,r}(o) > 0\} - \Phi\{\mathbf{F}_t^{s,r}(o) = 0\}), \quad (6)$$

to take non-historical entities into account, where λ is a hyperparameter and $\Phi\{\beta\}$ is the indicator function that returns 1 if β is true, and 0 else. Thus, $\mathbf{Z}_t^{s,r}(o)$ represents whether a quadruple is historical or non-historical event.

CENET learns the historical and non-historical dependencies based on the input. CENET employs a copy mechanism based learning [18] strategy to capture various types of dependencies, focusing on two aspects: the similarity score vector between the query and the set of entities, and the frequency information associated with the query, which is leveraged through the copy mechanism. The copy mechanism allows the model to directly incorporate relevant entities from the input into the output, enhancing its ability to generate accurate predictions.

CENET generates latent context vectors, $\mathbf{H}_{his}^{s,r}$ and $\mathbf{H}_{nhis}^{s,r} \in \mathbb{R}^{|\mathcal{E}|}$ which scores the historical and non-historical dependencies of different object entities by

$$\begin{aligned} \mathbf{H}_{\text{his}}^{s,r} &= \underbrace{\tanh\left(\mathbf{W}_{\text{his}}(\mathbf{s} \oplus \mathbf{r}) + \mathbf{b}_{\text{his}}\right) \cdot \mathbf{E}^{\mathrm{T}}}_{\text{similarity score between query and E}} + \mathbf{Z}_t^{s,r}, \\ \mathbf{H}_{\text{nhis}}^{s,r} &= \tanh\left(\mathbf{W}_{\text{nhis}}(\mathbf{s} \oplus \mathbf{r}) + \mathbf{b}_{\text{nhis}}\right) \cdot \mathbf{E}^{\mathrm{T}} - \mathbf{Z}_t^{s,r}, \end{aligned} \quad (7)$$

where $\oplus$ is the concatenation operator, and $\mathbf{W}$ and $\mathbf{b}$ are the trainable parameters for the historical and non-historical components, respectively. After applying the $\tanh$ activation function, the result is multiplied by the entity embeddings $\mathbf{E}$ to produce a vector of size $|\mathcal{E}|$, where each element corresponds to an entity $o' \in \mathcal{E}$ and the query q.

In the copy mechanism, $\mathbf{Z}_t^{s,p}$ is used to either add or remove information based on whether the entity is historical (his) or non-historical ($nhis$). This operation adjusts the index scores, increasing their values without affecting the gradient update. Consequently, $\mathbf{Z}_t^{s,p}$ helps $\mathbf{H}_{his}^{s,r}$ and $\mathbf{H}_{nhis}^{s,r}$ focus more on historical and non-historical entities, respectively.

To predict objects, CENET takes the softmax of the latent context vectors, $H_{his}^{s,p}$ and $H_{nhis}^{s,p}$, which is used as the predicted probabilities

$$\mathbf{P}_t^{s,r} = \frac{1}{2}\{\text{softmax}(\mathbf{H}_{his}^{s,r}) + \text{softmax}(\mathbf{H}_{nhis}^{s,r})\} \quad (8)$$

over all the object entities. By applying the softmax function to transform the values into probabilities, the entity with the highest probability can be interpreted as the one most likely predicted object.

The CENET model then uses cross-entropy loss

$$\mathcal{L}^{ce} = -\sum_{q} \log\{\frac{exp(\mathbf{H}_{his}^{s,p}(o_i))}{\sum\limits_{o_j \in \mathcal{E}} exp(\mathbf{H}_{his}^{s,p}(o_j))} + \frac{exp(\mathbf{H}_{nhis}^{s,p}(o_i))}{\sum\limits_{o_j \in \mathcal{E}} exp(\mathbf{H}_{nhis}^{s,p}(o_j))}\}, \quad (9)$$

with the goal to distinguish the ground truth entity from others.

2) Historical contrastive learning: To avoid that the model mainly uses historical data that is more frequent when predicting, CENET also employs so called *historical contrastive learning* that trains contrastive representations of queries to identify a small number of highly correlated entities at the query level.

To prevent the model from relying too much on the more frequent historical data during prediction, CENET incorporates *historical contrastive learning*, which trains contrastive representations of queries to focus on a select set of highly correlated entities at the query level. The training is divided into two parts, in the first stage I_q is introduced. Its purpose is to signify whether the object o for the query (s, r, o, t) is in $\mathbf{H}_t^{s,r}$ which then yields a 1, otherwise 0. In the second stage, a binary classifier is trained to predict the value of a boolean scalar for the given query q, that is used to identify if the query is historical or non-historical.

Stage 1 - Learning Contrastive Representations. In the first stage of training, the model learns contrastive representations of queries by minimizing a supervised contrastive loss, which uses the condition that I_q is positive as a training criterion. This loss function maximizes the separation of representations for different queries in the semantic space. For a given query q, let $\mathbf{v}_q$ denote the embedding vector, it is then calculated as

$$\mathbf{v}_q = \text{MLP}(\mathbf{s} \oplus \mathbf{r} \oplus (\tanh \mathbf{W}_F \mathbf{F}_t^{s,r}), \quad (10)$$

where an MLP is used to normalize the query's information and $\mathbf{W}_F \in \mathbb{R}^{d \times |\mathcal{E}|}$ is a trainable parameter.

The set of queries $Q(q)$ in the minibatch M except q itself whose boolean labels are the same as I_q are calculated

$$Q(q) = \bigcup_{m \in M \setminus \{q\}} \{m \mid I_m = I_q\}. \quad (11)$$

The supervised contrastive loss is then calculated as

$$\mathcal{L}^{sup} = \sum_{q \in M} \frac{-1}{|Q(q)|} \sum_{k \in Q(q)} \log \frac{\exp(\mathbf{v}_q \cdot \mathbf{v}_\mathbf{k}/\tau)}{\sum_{a \in M \setminus \{q\}} (\mathbf{v}_q \cdot \mathbf{v}_\mathbf{a}/\tau)}, \quad (12)$$

where $\tau \in \mathbb{R}^+$ is so called a temperature parameter set to 0.1. The goal is to bring representations of same historical class closer to each other and vice versa. $\mathcal{L}^{sup}$ and $\mathcal{L}^{ce}$ are trained simultaneously, yielding a final loss function

$$\mathcal{L}^{tot} = (1 - \nu)\mathcal{L}^{ce} + \nu\mathcal{L}^{sup}, \quad (13)$$

where $\nu \in [0, 1]$ is a hyperparameter weighting the influence of the loss functions.

Stage 2: Training binary classifier. In the second stage, CENET freezes all weights corresponding to the embeddings $\mathbf{E}$, $\mathbf{R}$ and the encoders in the first stage. It then sends $\mathbf{v}_q$ to a linear layer to train a binary classifier with ce-loss. The classifier is then trained to determine whether the missing object entity for the query q is historical or non-historical. To achieve this, a boolean mask vector

$$\mathbf{B}_t^{s,r}(o) = \Phi\{o \in H_t^{s,r} = \hat{I}_q\} \quad (14)$$

is created to identify the relevant type of entities based on the predicted $\hat{I}_q$ and whether $o \in H_t^{s,r}$ holds true. When this condition holds true, the probabilities of those entities are increased, putting more attention to them; otherwise, the probabilities are decreased.

3) Inference stage: To utilise the predicted classes, CENET masks the predicted probabilities according to whether they are predicted as nonhistorical or historical, which yields the final probability vector

$$\mathbf{P}(o|s,r,\mathbf{F}_t^{s,r}) = \mathbf{P}_t^{s,r} \odot \mathbf{B}_t^{s,r}, \tag{15}$$

where $\odot$ denotes the Hadamard product and the prediction for the most likely object can be extracted as

$$\hat{o} = \underset{o}{\operatorname{argmax}}\, \mathbf{P}(o|s,r,\mathbf{F}_t^{s,r}). \tag{16}$$

If the classifier is inaccurate, this may negatively impact the masking. Thus, it is also proposed to use softmax on the mask vector to yield a more convincing distribution as to obtain

$$\bar{\mathbf{B}}_t^{s,r} = \operatorname{softmax}(\mathbf{B}_t^{s,r}). \tag{17}$$

This allows for objects not of the historical class suggested by the classifier to predicted if the base model is supremely confident in them.

IV. Methodology

RE-GCN [3] performs worse on queries which have non-historical entities as answers than historical ones. CENET [5] sought to alleviate this by introducing a weighting during training to train two different base predictors to separately capture the probability distributions of historical and non-historical entities in combination with contrastive learning. The base predictors utilised in CENET $\mathbf{H}_\cdot^{s,r}$ (7) are simple models, consisting only of a single linear and a tanh operation. In contrast, RE-GCN is a more intricate model and could thus enable the capture of more complex temporal dynamics. Motivated by this, we sought to adapt and implement RE-GCN as a base predictor in CENET framework in order to improve the performance of RE-GCN on non-historical queries. An overview of the architecture is depicted in Figure 2.

A. RE-GCN as a Base Predictor

To adapt RE-GCN as a base predictor, we utilise the existing output scores over objects and add the history indicator vector $Z_t^{s,r}$ for the given query $(s,r,?)$. Thus, two the new base predictors are obtained as

$$\begin{aligned} \mathbf{H}_{his}^{s,r} &= \mathrm{REGCN}(s,r) + \mathbf{Z}_t^{s,r} \\ \mathbf{H}_{nhis}^{s,r} &= \mathrm{REGCN}(s,r) - \mathbf{Z}_t^{s,r} \end{aligned} \tag{18}$$

where both instances of RE-GCN share the embeddings of the subjects, relations and objects, but have different internal weights. Thus, each base predictor encodes different behavior depending on whether an object has been seen in combination with the query $(s,r,?)$ in the past. To combine the predictions from the two models, we adopt the approach from (8) with a slight change. The predictions from our model, before masking, take the form

$$\mathbf{P}_t^{s,r} = \alpha \operatorname{softmax}(\mathbf{H}_{his}^{s,r}) + (1-\alpha)\operatorname{softmax}(\mathbf{H}_{nhis}^{s,r}), \tag{19}$$

where $\mathbf{H}_{his}^{s,r}, \mathbf{H}_{nhis}^{s,r}$ are defined as in (18) and $\alpha \in [0,1]$ is a hyperparameter to determine whether to put more weight on the historical or non-historical base predictor. This allows for the predictions to be more based on one base predictor or the other. Our motivation for adding this hyperparameter was that in the ablation study as presented in [5], the model performs poorly using only the non-historical base predictor in comparison to the only historical. However, when combined they influence the results equally, which seems at odds at with the results from the ablation study.

B. Adapting the Binary Classifier

We employ the same classifier architecture as utilized in CENET. This consists of the encoder (10) where a single linear layer is used and a binary classifier, which consists of a two linear layers and a sigmoid function at output to normalize the results to between zero and one. The boolean mask vector $\mathbf{B}_t^{s,r}$ from (14) is then produced. One difference compared to CENET is that we use different embeddings of the subjects, relations and objects for the contrastive classifier as opposed to CENET where they are shared with the base predictors. This also means they are not frozen when training the classifier. We initially tested sharing them, but found that the classifier performed worse (about 10% less classification accuracy on ICEWS14). We reason that since RE-GCN is a much more complex base predictor than the one employed by CENET, its resulting embeddings may be more subtle for the classifier to work with.

To obtain the final predictions, we allow for the three different masking alternatives presented in CENET, none (skipping the masking step altogether), hard(15), and soft(17).

V. Implementation

The proposed model extends the RE-GCN model, with adjustments to the input and output handling that allow for significant code reuse. As a result, portions of the CENET architecture were rebuilt upon the RE-GCN code.

A. Data processing

Both RE-GCN and CENET have scripts for how they pre-process the data, parsing the raw triples from text files into arrays which are later used for training. The main difference between them is that in CENET the frequency vector $\mathbf{F}_t^{s,r}$ (5), is created in the pre-processing stage, but the mapping between indices to variables are the same for both models. Thus, the same pre-processing of the datasets have been reused.

There are six popular datasets for TKGs, where this proposed method is evaluated on ICEWS18 [19]. The dataset originate from the Integrated Crisis Early Warning System (ICEWS) and consist of interactions between socio-political actors. The characteristics of the dataset are displayed in Table II, where the number of different entities, relations and all the facts/quadruples for the training, validation and test set are given.

B. Evaluation and metrics

We evaluate our model using the same evaluation metrics and filtering methods used in [3] which comes naturally from

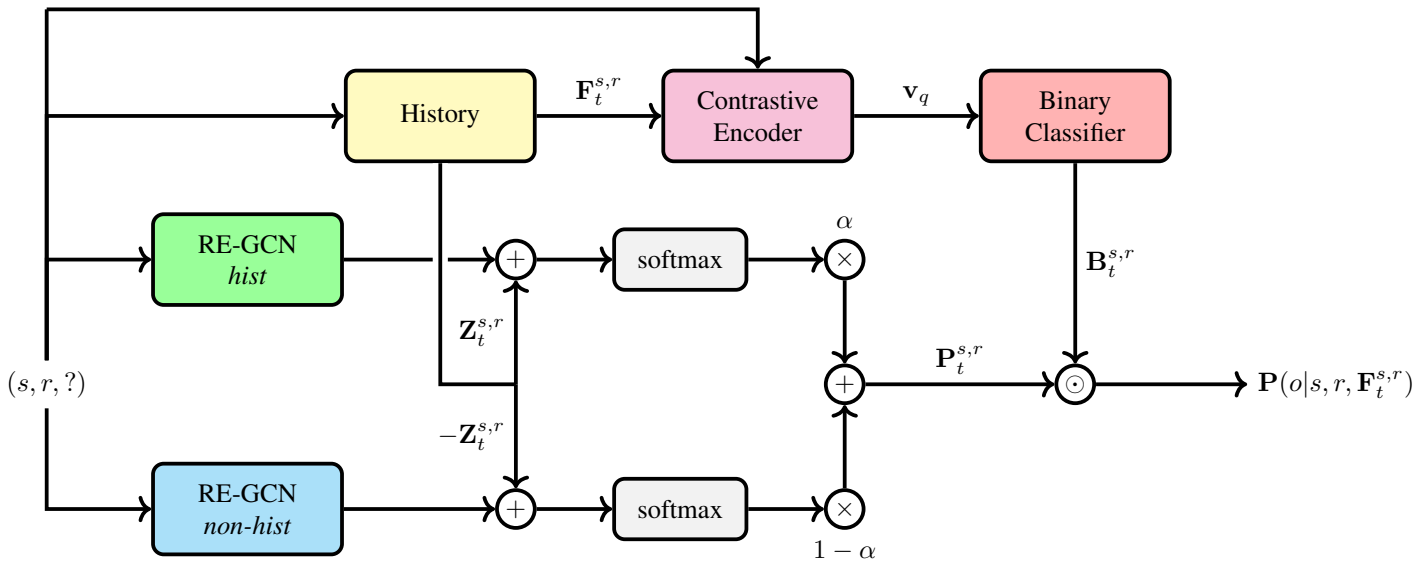


Fig. 2. An overview of the designed model architecture with RE-GCN as base predictors. The figure depicts the hard-mask case.

TABLE II
STATISTICS OF THE ICEWS18 DATASET [5].

Dataset	Entities	Relations	Train	Val	Test
ICEWS18	23 033	256	373 018	45 995	49 545

having RE-GCN as our baseline and upon which our approach heavily modifies.

Metrics: We use Mean Reciprocal Rank (MRR) and Hits@k for $k = 1, 3, 10$ which are ways to evaluate how well the network is at predicting entities or relations. Further, we use a raw version when evaluating.

Filters: For each test triple (s, r, o), the model estimates object probabilities for the query $(s, r, ?)$, designating these as *corrupted triples*. These probabilities are then sorted, and the rank of the correct entity is noted. This process is then repeated by instead predicting the subject for the query $(?, r, o)$. The Mean Reciprocal Rank (MRR) is calculated as the average of the reciprocal ranks for all test queries, while Hits@k, within the top k, measures the proportion of correctly ranked entities [16].

There are several methods available for evaluation, each with a different approach to handling corrupted triples. In this report, we present results using *raw* metrics. In contrast, time-aware filters remove only quadruples that share the same timestamp as the test query, allowing historical facts to remain, which improves prediction accuracy.

C. Hyperparameter search

To identify the best hyperparameter settings, we conducted a hyperparameter search over λ, α, and ν. We selected these parameters as we considered them most likely to have the most impact on the architecture's performance and are novel in combination with RE-GCN. Therefore, we used the model configurations and hyperparameters reported as optimal in the RE-GCN authors [3] for the base-predictors. Limited by time and computational resources, we conducted a smaller-scale hyperparameter search. Additionally, to speed up the training process, we reduced the ICEWS18 dataset to 80% of its original size. Our search consisted of evaluating all combinations of $\lambda \in \{0.5, 2, 4, 16, 32\}$, $\alpha \in \{0.01, 0.05, 0.1, 0.5, 0.9\}$ and $\nu \in \{0.25, 0.5, 0.75\}$. From this set, the best performing combination was found to be $\lambda = 32, \alpha = 0.1, \nu = 0.75$.

VI. EXPERIMENTAL EVALUATION

TABLE III
EVALUATION METRICS FOR ENTITIES - ICEWS18 20 EPOCHS USING GT, TAKING THE MEAN OF FIVE MODELS

		RE-GCN Model	No mask	Hard mask	Soft mask
Raw	MRR	**0.307**	0.299	0.249	0.298
	Hits@1	**0.201**	0.193	0.167	0.193
	Hits@3	**0.349**	0.340	0.285	0.340
	Hits@10	**0.516**	0.513	0.410	0.505
Hist.	MRR	0.411	**0.412**	0.328	0.411
	Hits@1	**0.280**	0.278	0.227	0.278
	Hits@3	0.473	**0.477**	0.380	0.475
	Hits@10	0.671	**0.677**	0.527	0.674
Non-Hist.	MRR	**0.115**	0.089	0.102	0.090
	Hits@1	**0.056**	0.035	0.055	0.036
	Hits@3	**0.118**	0.087	0.109	0.090
	Hits@10	**0.230**	0.192	0.194	0.193

VII. DISCUSSION

As in shown in Table III, our proposed model does not yield any significant improvement over RE-GCN over all queries nor non-historical case which we especially sought to improve with our additions. While it seems that our model might actually decrease performance this category, this could also be due to different initialization of the weights when

training the model. Thus, we find it most likely that our model neither improves or decreases performance. With CENETs base predictors, the addition of the contrastive approach drastically improves performance. We hypothesize that this improvement does not manifest with RE-GCN because of the its heightened ability to capture temporal dependencies compared to the simple base predictor employed by CENET. Because of RE-GCNs autoregressive nature, it already encodes a notion of temporal dynamics and history, which is lacking from CENETs base predictors. Thus, the historical information we provide to improve the predictions may be unnecessary. Furthermore, possible benefits from including this information may be counteracted by the increased complexity of model, which renders it harder to train and more prone to overfitting.

This also offers a plausible explanation for why the inclusion of the classifier does not offer any substantial performance increase. Since the classifier tries to encode historical information, it does not in effect have any more information than the base RE-GCN model. Conjoined with this, is that the classifier is inherently less complex and specific than RE-GCN. In principle, the soft mask should allow for the best of both RE-GCN and the classifier to be combined. However, since the scores given by RE-GCN are uncalibrated (the softmax scores do not necessarily encode the same type of behavior as an actual probability), the bayesian interpretation of the soft mask is not necessarily valid. Furthermore, if RE-GCN already has the correct historical class correct for its top objects, those specific predictions may be still be ranked incorrectly, and then the classifier cannot assist.

VIII. Conclusion

We sought to explore avenues to improve the RE-GCN model's ability to predict future events within temporal knowledge graphs, especially focusing on improving its performance on non-historical queries. To address this challenge, we incorporated a contrastive learning framework inspired by the CENET model [5]. This framework introduced a two step prediction setup, where two RE-GCN (historical and non-historical) served as the base predictor, while a classifier utilizing contrastive training was added to separately capture historical and non-historical information. Our intention was to enable RE-GCN to handle more complex, diverse temporal patterns beyond those strictly rooted in historical interactions.

The two RE-GCN models were modified with two bias vectors, which increased and decreased prediction scores for the relevant entities. The contrastive classification mechanism was designed to selectively adjust the focus on historical- or non-historical relationships by masking a subset from the base predictions. We applied hyperparameters to balance these processes, allowing us to experiment with optimal configurations. However, while our model demonstrated a satisfactory performance on historical predictions, we were unable to capture improved results for non-historical queries.

This project's findings underscore the complexity of predicting non-historical events in TKGs. Our experimental results on the ICEWS18 dataset suggest that models primarily grounded in historical learning may struggle to generalize to cases with absent historical context. Further investigation of our model should explore testing on diverse datasets and applying more extensive hyperparameter tuning.

IX. Future work

We believe it could be promising to explore uncertainty quantification to focus on trying to quantify when a query is likely to correctly predict the entity. This information could then be used downstream to guide decision making, for example investments. It could also be of interest to successively simplify the base predictors to see at what complexity they perform best and to study how the contrastive approach contributes (or not).

X. Takeaways, thoughts and recommendations

- If you are working with ML models, test early if you are able to install and run the models. Look at how long they take to train as this could limit how fast you are able to iterate. For the case of temporal knowledge graph extrapolation, training and testing can be fairly time and computation heavy endeavors, so it can be wise to devise a reduced scheme to allow for quick hints of whether an approach could be promising.
- We recommend to extend on someone else's work rather than improve as to not get too hung up on raw results and trying to improve with multiple changes. Extending on someone else's work might be more rewarding as an improvement is not necessary.
- Conduct a comprehensive literature review with the aim of formulating a clear and focused research question. Reflect on whether the question is inherently compelling, ensuring it remains valuable and thought-provoking, even if the results turn out to be negative.

Acknowledgments

The authors would like to thank Georg Schuppe, Balazs Toth, and the rest of the SEB team for their support and guidance throughout the development of this work. They not only served as the foundation of our research but also reminded us, much like the systems we studied, that anticipating what comes next—whether in data or life—is both an art and a science.

Also, we would like to thank Julia Gastinger (PhD. student at University of Mannheim) for providing guidance early in our project to give more context on the TKG landscape.

References

[1] B. Sanchez-Lengeling, E. Reif, A. Pearce, and A. B. Wiltschko, "A gentle introduction to graph neural networks," *Distill*, 2021, https://distill.pub/2021/gnn-intro.

[2] L. Cai, X. Mao, Y. Zhou, Z. Long, C. Wu, and M. Lan, "A survey on temporal knowledge graph: Representation learning and applications," 2024. [Online]. Available: https://arxiv.org/abs/2403.04782

[3] Z. Li, X. Jin, W. Li, S. Guan, J. Guo, H. Shen, Y. Wang, and X. Cheng, "Temporal knowledge graph reasoning based on evolutional representation learning," 2021. [Online]. Available: https://arxiv.org/abs/2104.10353

[4] W. Jin, M. Qu, X. Jin, and X. Ren, "Recurrent event network: Autoregressive structure inference over temporal knowledge graphs," 2020. [Online]. Available: https://arxiv.org/abs/1904.05530

[5] Y. Xu, J. Ou, H. Xu, and L. Fu, "Temporal knowledge graph reasoning with historical contrastive learning," 2022. [Online]. Available: https://arxiv.org/abs/2211.10904

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[7] Z. Han, P. Chen, Y. Ma, and V. Tresp, "Explainable subgraph reasoning for forecasting on temporal knowledge graphs," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=pGIHq1m7PU

[8] H. Sun, J. Zhong, Y. Ma, Z. Han, and K. He, "TimeTraveler: Reinforcement learning for temporal knowledge graph forecasting," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 8306–8319. [Online]. Available: https://aclanthology.org/2021.emnlp-main.655

[9] Y. Xia, D. Wang, Q. Liu, L. Wang, S. Wu, and X. Zhang, "Chain-of-history reasoning for temporal knowledge graph forecasting," 2024. [Online]. Available: https://arxiv.org/abs/2402.14382

[10] Z. Ding, H. Cai, J. Wu, Y. Ma, R. Liao, B. Xiong, and V. Tresp, "zrllm: Zero-shot relational learning on temporal knowledge graphs with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2311.10112

[11] R. Luo, T. Gu, H. Li, J. Li, Z. Lin, J. Li, and Y. Yang, "Chain of history: Learning and forecasting with llms for temporal knowledge graph completion," 2024. [Online]. Available: https://arxiv.org/abs/2401.06072

[12] R. Liao, X. Jia, Y. Li, Y. Ma, and V. Tresp, "GenTKG: Generative forecasting on temporal knowledge graph with large language models," in *Findings of the Association for Computational Linguistics: NAACL 2024*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 4303–4317. [Online]. Available: https://aclanthology.org/2024.findings-naacl.268

[13] Y. Li, S. Sun, and J. Zhao, "Tirgn: Time-guided recurrent graph network with local-global historical patterns for temporal knowledge graph reasoning." in *IJCAI*, 2022, pp. 2152–2158.

[14] J. Gastinger, C. Meilicke, F. Errica, T. Sztyler, A. Schuelke, and H. Stuckenschmidt, "History repeats itself: A baseline for temporal knowledge graph forecasting," 2024. [Online]. Available: https://arxiv.org/abs/2404.16726

[15] M. Zhang, Y. Xia, Q. Liu, S. Wu, and L. Wang, "Learning latent relations for temporal knowledge graph reasoning," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 12 617–12 631. [Online]. Available: https://aclanthology.org/2023.acl-long.705

[16] J. Gastinger, T. Sztyler, L. Sharma, A. Schuelke, and H. Stuckenschmidt, "Comparing apples and oranges? on the evaluation of methods for temporal knowledge graph forecasting," in *Machine Learning and Knowledge Discovery in Databases: Research Track*, D. Koutra, C. Plant, M. Gomez Rodriguez, E. Baralis, and F. Bonchi, Eds. Cham: Springer Nature Switzerland, 2023, pp. 533–549.

[17] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," 2020. [Online]. Available: https://arxiv.org/abs/2002.05709

[18] J. Gu, Z. Lu, H. Li, and V. O. K. Li, "Incorporating copying mechanism in sequence-to-sequence learning," 2016. [Online]. Available: https://arxiv.org/abs/1603.06393

[19] E. Boschee, J. Lautenschlager, S. O'Brien, S. Shellman, J. Starz, and M. Ward, "ICEWS Coded Event Data," 2015, we are unable to provide the story text from which events are extracted or the URLs due to licensing restrictions. [Online]. Available: https://doi.org/10.7910/DVN/28075

[20] H. Ramsauer, B. Schäfl, J. Lehner, P. Seidl, M. Widrich, T. Adler, L. Gruber, M. Holzleitner, M. Pavlović, G. K. Sandve, V. Greiff, D. Kreil, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, "Hopfield networks is all you need," 2021. [Online]. Available: https://arxiv.org/abs/2008.02217

[21] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2016. [Online]. Available: https://arxiv.org/abs/1409.0473

APPENDIX A
RE-GCN

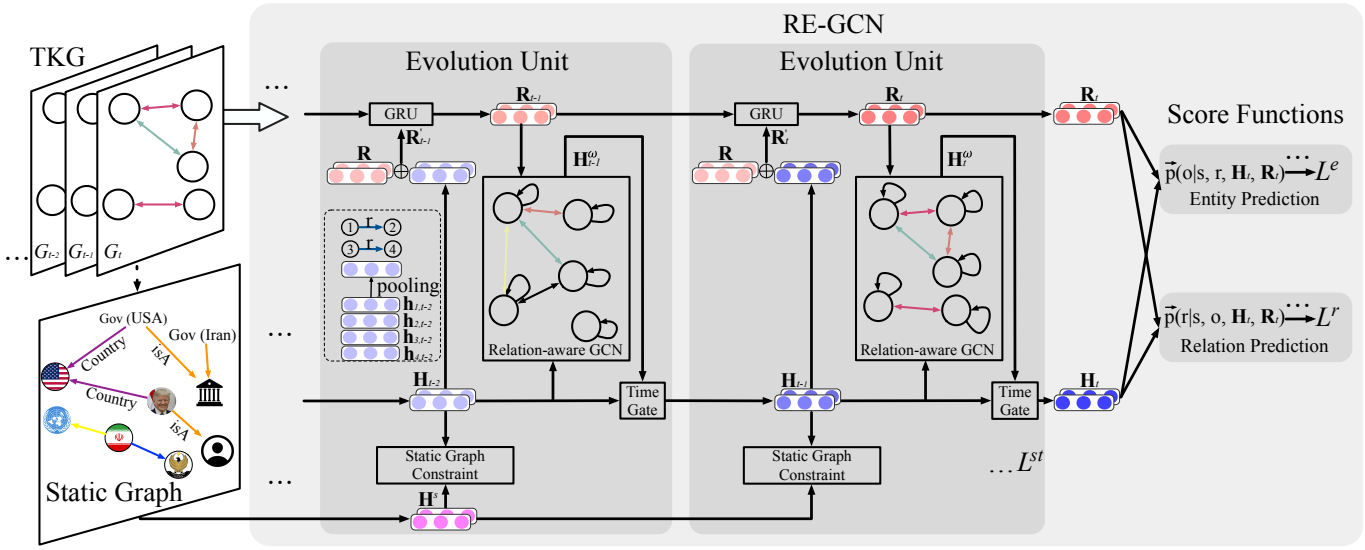


Fig. 3. Architecture of RE-GCN — an illustrative diagram of the proposed RE-GCN model for temporal reasoning at timestamp *t+1* [3].

APPENDIX B
CENET

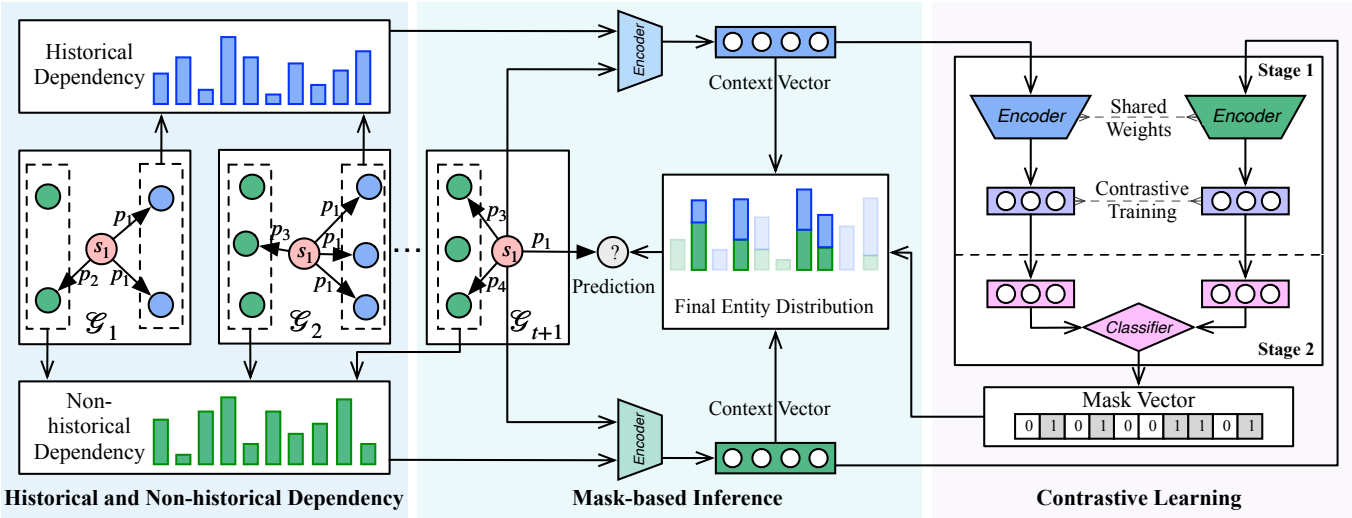


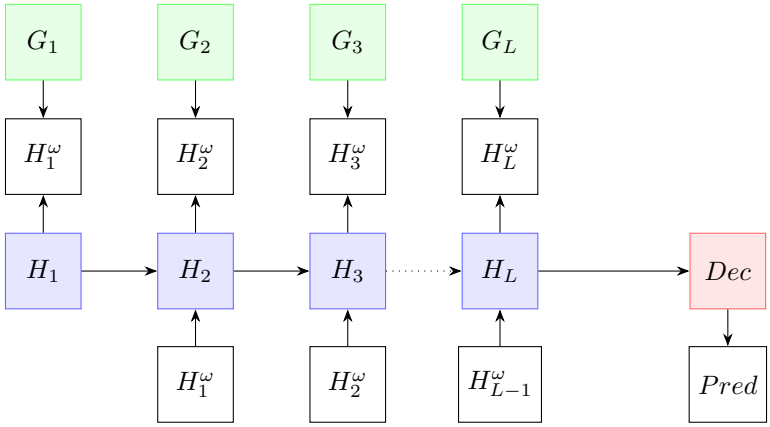
Fig. 4. Architecture of CENET — an illustrative diagram of the proposed CENET model for temporal reasoning at timestamp *t+1* [3].

APPENDIX C
HOPFIELD

During our literature review, we came across the paper Hopfield Networks Are All You Need [20], which suggests that Hopfield networks could replace RNN components such as GRUs or LSTMs to achieve better results. We found this idea interesting and decided to investigate whether substituting the GRU component with a Hopfield network would yield improved performance. The modern Hopfield network is capable of storing exponentially many patterns and retrieving them with a single update, demonstrating an equivalence to the attention mechanism found in transformers. This motivated us to explore whether the patterns stored by the Hopfield network would outperform the hidden states retained by the GRU.

We ran some tests on swapping out GRU for (modern) Hopfield, but isn't as trivial as one would think. Because of the lack of an explicit hidden vector as input, it isn't possible to do a direct swap in our case, since the single layer of GRU is tightly interwoven in RE-GCN structure.

APPENDIX D
ATTENTION



This is a simplified, unrolled version of the RE-GCN architecture from a slightly different perspective compared to fig.Figure 3, where, for simplicity, we omit the relational component and the static graph. $H_1, H_2, H_3, ..., H_L$ denotes an arbitrary time sequence of length L. $G_1, G_2, G_3 ..., G_L$ are the mutual graphs, for each time step. H_1 denotes the entity embedding matrix, H_i, where $i \neq 1$ refers to the hidden embedding matrices, and H^ω represents the embedding matrix generated by the RGCN. The arrows from H_{i-1}^ω and H_{i-1} to H_i represent the corresponding terms in the time gate operation.

As shown, the hidden embeddings are produced in a manner very similar to a recurrent neural network (RNN), with the key difference being that H_{i-1}^ω and H_{i-1} share weights. Moreover, this setup forms an autoregressive RNN, where H_i is dependent on the previous output H_{i-1}^ω.

A central idea behind this architecture is to optimize the network such that it generates a final hidden representation H_L that encapsulates sufficient information about the entire sequence, enabling the decoder to make accurate predictions. However, when dealing with longer sequences, this becomes challenging. Valuable information may reside further back in the sequence, and capturing all relevant autocorrelations is of interest. This issue is reminiscent of the work by Bahdanau et al. (2016) [21], where the translation of long sequences was significantly improved through the introduction of the alignment/attention mechanism. Interestingly, their approach also led to improvements for shorter sentences, which is more closely aligned with our task at hand.

We implemented a version of the attention/alignment mechanism from Bahdanau et al. to calculate a context matrix, with the goal of capturing more relevant parts of the hidden sequences generated by RE-GCN. In this approach, the final hidden matrix (input to the decoder) was replaced by the context matrix, which holds context vectors for each entity.

The following equations describes how the context embedding for a single entity is calculated:

$$e_j = \mathbf{v}^\mathbf{T} \tanh(\mathbf{W}\mathbf{h}_j) \tag{20}$$

$$\alpha_j = \frac{\exp(e_j)}{\sum_{k=1}^{L} \exp(e_k)} \tag{21}$$

$$c = \sum_{j=1}^{L} \alpha_j \mathbf{h_j} \tag{22}$$

$\mathbf{W}$ and $\mathbf{v}$ are learnable. We compute the context vectors for all entities at the same time, using matrix multiplication, ensuring that the weights are shared across all entities. This process results in the final context matrix. Below is a toy example of how the context might weigh some hidden representations in a sequence of $L = 3$

$\mathbf{h}$	α
$\mathbf{h_1}$	0.66 (α_1)
$\mathbf{h_2}$	0.01 (α_2)
$\mathbf{h_3}$	0.33 (α_3)

Unfortunately this implementation has not yielded any significant improvements so far.

A quick inspection of the similarity between the hidden embeddings within the same sequence revealed very small differences. This is a problem, because attaching many similar embeddings with attention weights and then adding them should not capture any underlying structure of the sequence.

APPENDIX E
PROGRESS DIARY

A. Week 36

Summary: This week we worked mainly on the startup of the project having multiple meetings and making the project plan. We also had a initial meeting with SEB where we discussed the outline of the project and scheduled upcoming meetings where we also got some resources to look into. Throughout these meetings, we collaborated on defining our approach and establishing how we will work together moving forward.

Next week we plan on starting the work of implementing the RE-NET and begin our literature studies, which also will be used as the source of other models we would like to explore.

Communication with group 18: We sat up a discord channel where we can share information as well as have our meetings. We discussed having meetings every Thursday after our meeting with SEB to discuss the projects. All members were added to this channel where we have different pages to communicate different topics.

Members: *This week have mainly been used for startup of the project and read a little about the project, for coming weeks we will individually write a sentence each of our work.

- **Alexander:** 10h (including familiarize with theory and relevant material like video lectures and reports, and note taking.)
- **Emilia:** 5h
- **Maximilian:** 5h Mostly meetings, planning and reading the RE-NET paper.
- **Sixten:** 5h

B. Week 37

Summary: Work this week mostly consisted of literature review and getting RE-Net (a model selected as a baseline by our project supervisors) to run.

We got RE-Net up and running in colab after a few versioning issues due to the code using some functions whose behaviour has changed in later versions of Python. While we did get it running on modern Python versions eventually (by augmenting the code somewhat), we settled on downgrading to the versions used by the original authors in order to minimise the risk that the code is not working as intended by the authors.

Literature review has been coming along well, but since none of us have any prior familiarity with graph NNs, a lot of the reading has been focused on understanding the fundamentals. Georg at SEB sent us a talk of his as an intro to GNNs, which was a great starter. Beyond that, we have watched lectures from a Stanford course on machine learning with graphs and read a few survey articles to get a better overview of the landscape.

During our meeting with SEB this week, we focused on discussing RE-Net implementation details and what parts of it (or other models) could be promising to augment to improve performance.

The other group seems to have wandered along similar paths this week, and we discussed our different approaches for getting RE-Net running on colab.

Next week:

- Study RE-Net article in more detail over the weekend and then discuss to make sure we all understand it properly.
- Discuss what component we would like to try changing or swapping out.
- Try to swap out said component by making a simple change (not necessarily an improvement) to gauge the difficulty of working with the code.
- Look into other models.

Communication with group 18: This week we had a meeting where we discussed some of the code implementations, and also helped and shared our code of how to run the network. We also discussed a problem we found with the code, namely it crashing after the 10th epoch. During this week we have also discussed a little about approaches to the projects, such as possible extensions.

- Alex: 10h in total. 2h stanford lectures, 3h literature research, 3h looking into RE-NET code, 2h meetings.
- Max: 2h literature, 1h Stanford lecture on Machine Learning with Graphs, 2h meetings, 4h RE-NET code setup. Total: 9h
- Emilia: 8h in total, 1h watching a video about graph neural networks, 3h literature research, 1h studying the RE-NET code, 1h studying the RE-NET paper and about 2h meetings.
- Sixten: 12h. 2h meetings, 2h literature. 8h RE-Net and alternative model code setup and testing.

C. Week 38

Summary: This week was spent mostly on literature review on specific models but also on the field of TKGs as a whole. There was also some effort into making RE-NET work but with the meeting with SEB on Thursday they re-focused us on picking a direction that we want to pursue and not bother making RE-NET work if it's annoying to work with and just pick another model that we find interesting.

Next week: This lead us to set a goal to decide on one model we want to improve by Monday next week by reading literature and training the models for testing.

Communication with group 18: The other group did not respond to meeting request this week.

- Alex: 12h in total. Browsing papers (2h), reading papers (3h), meetings (3h), standford lectures incl notes (2h), stairing at RE-NET code (2h).
- Sixten: 14h. Mostly spent getting model training set up (including a fair amount of debugging for RE-Net), some literature review and meetings.
- Max: 4h. Sick mostly, but did run some code tests and read some articles.
- Emilia: 11h. Reading papers and doing literature review , studying RE-NET code and meetings.

D. Week 39

Summary: This week, we decided on a direction for the network to work with and settled on RE-GCN, as it is commonly referenced in modern papers in the field, closely resembles the initially suggested RE-NET (which we have already spent considerable research time on), and aligns with some creative ideas we have, such as utilizing a Hopfield network. However, we are still exploring other potential improvements that could be made to RE-GCN by implementing small modifications, such as pruning or substituting components. We also established a system for reporting individual experiments to track and organize results for discussion at the upcoming Monday meeting.

1) Swapping out GRU for Hopfield: Ran some tests on swapping out GRU for (modern) Hopfield, but isn't as trivial as one would think. Because of the lack of an explicit hidden vector as input, it isn't possible to do a direct swap in our case, since the single layer of GRU is tightly intervowen in RE-GCN structure. Two simple approaches we tried were however:

1) Running the memory vector of GRU through a hopfield network before getting passed along. The hope would be that this would in some sense "clarify" the memory by making more similar to a pattern the Hopfield network had seen before. This seems to have minimal effect on performance, which is somewhat interesting in its own right.
2) Concatenating the memory vector with the original input vector into one big vector, which then was used as input by the Hopfield network. This approach does (to my [Sixten] best knowledge) not really make sense from a structural point of view. The results also showcase this, with results on MRR and Hit@k decreasing quite a bit when comparing to an equivalently trained GRU network.

To integrate Hopfield in a *proper* way, more structural changes would likely be required of the architecture. Further investigation will ensue!

2) Swapping out GRU for LSTM: I (Alex) conducted tests to determine if there were any differences between GRU and LSTM. The initial assumption was that LSTM might better capture more complex and long-term dependencies. Unlike GRU, LSTM produces two outputs: a hidden state and a cell state. The cell state represents long-term memory, while the hidden state represents short-term memory. Typically, the hidden state is used as the embedding when making predictions. However, due to the architecture of RE-GCN and my current understanding, it seemed interesting to experiment with using both LSTM outputs as input to the RGCN, as well as a combination of them. Each version of RE-GCN was trained for 10 epochs and compared.

- hidden state as input to RGCN
- cell state as input to RGCN
- sum of hidden state and cell state as input to RGCN

Results: There was no significant difference in performance between the GRU and LSTM. Both models and versions produced similar results across all metrics, suggesting they performed comparably in capturing the necessary sequence information from the historical embeddings.

Conclusion: These results lead me to question the relevance of using GRU/LSTM in this context. It's possible that the representations generated by the GRU and LSTM either fully capture the sequence of historical embeddings, or that both models only capture very short-term dependencies, the input itself might be inherently short-term dependent.

Added note: might gain some insight by comparing LSTM and GRU over longer sequences.

- Alex: 16h. Experimenting with the GRU, i.e. substitute with LSTM, zeroing different GRU inputs. Implementing attention-like structure similiar to Badanahu et al 2014 (12h), Reading relevant papers (1h), meetings (3h).
- Sixten: 12h. Reading up on Hopfield (2h), integrating Hopfield into code and various setup tasks (7h), meetings (3h).
- Max: 8. Meetings 2h, 2h literature, 4h code analysis
- Emilia: 10h, study and experiment on RE-GCN code 4h, trying to get dgl to work... 3h, meetings 3h

E. Week 40

Summary: This week we continued our work on the RE-GCN where we experimented with different changes, like last week. We have tried switching out the GRU component in the evolution unit with the Hopfield network. However, we haven't really come up with a satisfying solution that yields better results.

So in the beginning of the week, we discussed possible changes, where we think it would be interesting to experiment with the Time Gate, as well as the decoder where they use the ConvTransE and also looked at adding attention to the static graph embeddings. One thing the whole group agreed upon was that we all wanted to explore contrastive learning, taking inspiration of the network CENET, we have read about in the past weeks

During the week, Alexander tried a modification that would give the model more direct access to the static graph embeddings to see if it would change performance. The reasoning would be that it maybe doesn't make sense that the static embeddings are put through the time gate. It did not change performance.

There have also been several discussions of how to incorporate contrastive learning and taking components/using some of the architecture from CENET. Here we discussed a lot whether to add the contrastive ce-loss or add contrastive learning part, and we decided to study the contrastive learning further.

We have also looked a bit at changing the decoder to other ones but haven't gotten any better results doing that. Other than that, we have experimented some more with the code and also started writing the report.

Next week: So the goal for next week is to study contrastive learning further, looking into the attention part more, as well as writing half-time evaluation report and continue writing the report.

Communication with group 18: This week before the scheduled meeting we have with them, they stated that they didn't have the time to have a meeting.

- Alex: 15h total. Followed up on experimentation with attention (8h), brainstorming ideas on network improvement (2h), meetings (4h), planning how to integrate contrastive learning into RE-GCN (1h).
- Sixten: 8h. Meetings 4h. Losing hope in Hopfield 1h. Planning how to integrate contrastive learning into RE-GCN 3h.
- Max: 10h. Meeting 4h. Experimenting with RE-GCN 6h.
- Emilia: 9h. Meetings 4h, write report 2h, reread papers about contrastive learning and re-gcn 2h, playing around with re-gcn code 1h

F. Week 41

Summary: This week our focus was on getting started with our implementation of contrastive learning as inspired by CENET. We came to the conclusion that this could be done in two steps; first that we implement the learning of contrastive embedding for queries and implement the contrastive loss; second that we implement the classifier from CENET to predict whether a query should have a historical or non-historical answer. After a fair amount of coding, we have implemented the first part, although it took longer than expected, due to the CENET code being more intricate than we initially thought and having a very non-intuitive variable naming convention, which tripped us up. Some testing was done without the classifier in place, and the results are slightly worse than the non-augmented model. This was disparaging, but not entirely unexpected. Our hope is now that the addition of the classifier should increase the performance. However, we have noted that RE-GCN seems to very robust to substantial changes to the model, which is somewhat frustrating. Most changes we have tried to not change the performance much at all. We must also verify that our implementation of this loss is correct. We are unsure about this because of some intricacies in how RE-GCN indexes its relations.

To side-step this issue, we also spent some time looking into uncertainty quantification instead. The goal would then be not to increase the model's performance, but instead try to get it to quantify it's uncertainty in a useful way. For example, instead of outputting a single object as answer to a query, it could output a a set of objects which it predicts the true object should lie in with 90% probability. If for example the query was (*Côte d'Ivoire*, *criticise*, ?) it could a yield the set [*Togo*, *Benin*, *Banque de France*]. We tried doing this by using a method called Rapid Adaptive Prediction Sets, which is tailored for this sort of multiclass prediction task, but sets were far too big to be useful. For 90% confidence, they typically included around 300 possible objects. This hints towards that RE-GCN does not encode uncertainty well in its scoring vector.

Next week: Next week we expect to work a bit less due to exams. Sixten will look into relation indexing, Alex will finish up on some transformer testing, Max and Emilia will work on the classifier.

Communication with group 18: We discussed mostly about their new ideas as presented during the SEB meeting and how they came up with it. We were fascinated by their novel and elegant approach. We noted that their work seems more conceptual around how to utilise LLMs to aid predictions, while ours is more technical and focuses on architectural improvements.

- Alex: 10h tot. 4h meetings, 4h exploring contrastive learning/looking at code, 2 hour experiments (attention).
- Sixten: 17h. 4h meetings. 10h contrastive learning coding. 3h exploring uncertainty quantification.
- Max: 4 hours. 2h meetings. 2h running tests and looking at our newest code.
- Emilia: 8h, 2h meetings, 1h code, 5h writing report

G. Week 42

Summary: This week was kind of ignored as group members decided to focus on upcoming exams. **Next week:** Next week our goal is to resume our work and implement classifier and historical/non-historical weighting for the contrastive addon. **Communication with group 18:** None

- Alex:
- Sixten: None
- Max: None
- Emilia: None

H. Week 43

Summary: Most work this week has been to finalize the implementation of CENETs classifier/oracle to our contrastive re-gcn. The model seems to perform better if the classifier is included, but it is not a huge change and unclear if statistically relevant. It is unclear if our model at this point actually handles historical/non-historical better because we only look at overall accuracy. We are also cleaning up the code by adding everything as arguments so we can set up a testing framework for the weeks to come. We also tested a bit on different datasets. We were a bit low capacity this week because people were still having exams.

Next week: As we are approaching the draft report, the group wants the upcoming week to combine our efforts so that we have one unified codebase where we can turn our implementations on/off so we can perform some testing. Then the goal is to set up some additional metrics like historical/non-historical MRR (accuracy). Then we want to perform more exhaustive test like some form of grid search and other datasets so that we can gauge if we actually have some kind of performance upgrade. We also want to focus more on the report too.

Communication with group 18: None they were still on exams.

- Alex: 4h. Meeting and looking for potential changes to current architecture, brainstorming new ideas.
- Sixten: 6h. Meetings + getting the code to be able to run larger datasets without going OOM.
- Max: 15h - Focus on getting oracle working. Also a lot of testing with different configurations.
- Emilia: 8h - Meetings, running code, looking at the classifier

I. Week 44

Summary: This week the original plan was to begin testing the model, but we found some problems with the classifier that we have been troubleshooting. The classifier outputs only historical labels on all the queries and never nonhist. We have tried multiple approaches to see if the classifier is correctly coded, if there are other issues with our code or if it is just very bad.

We have also written more on the report and hope to have an almost finished one by the deadline of the draft report.

Next week: Next week, we hope to have finished the troubleshooting and start evaluating and testing the model on a bigger scale using so called *optuna* to find the best hyperparameters. We also plan to continue writing the report as to have as close to finished report as soon as possible (save for maybe some tests that needs to be done).

Communication with group 18: The other group canceled due to seminars and sickness. They are now evaluting the model and writing the report.

- Alex: 6 hours in total. 2 hours meeting, 2 hours looking at potential issues with current code, 2 hours of further trying to improve attention version.
- Sixten: 7h. Meetings + classifier work.
- Max: 10h - Found issue with classifier/oracle, working on understanding why by debugging.
- Emilia: 8h - 1hour meeting, 1hour code, 6hours writing report

J. Week 45

Summary: This week the work centered around polishing the code so we could conduct a grid search over the hyperparameters in order to then conduct our final testing and accelerated work on the report. Unfortunately we ran into a rather nasty bug when setting up the grid search where the code would crash during training without leaving any indication of where it failed. It turned out to be a tensor initialization issue (we had allocated memory for the tensor but not cleared its previous contents), leading to invalid gradients. When this issue was resolved, we started the grid search, which of the time of writing is still ongoing. Report work is ongoing, but the results and discussion sections have to be fleshed out considerably.

Next week: Continued hyperparameter search, then testing the model with the found results. Continued report writing and handing the draft report.

Communication with group 18: We discussed our different definitions of ”zero-shot” and spoke a bit on testing methodologies. We also discussed how we were each planning and working on our reports.

- Alex: 10h total - 4h meetings, 4h setting up and deploying vm, 2h final code polish before training.
- Sixten: 14h - 4h meetings, 10h code (mostly polishing and implementing grid search).
- Max: 10h - 4h meetings and collaboration + 6h debugging
- Emilia: 8h - 4h meetings, 4h writing report

K. Week 46

Summary: Our grid search finalized and we spent some time analyzing the results. We then re-trained a model of the full data with the hyper params we found to be best. We trained the model four times to have some statistical gauge. Unfortunately the results were not as we had hoped and it seems like our model actually performs worse on non historical which we thought we would improve on. Then we started finalizing the draft report which we have submitted. **Next week:** Next week focus is on working more on the report, also starting to prepare a presentation for SEB. We might also investigate on why our method does not work and why it gives worse results than vanilla re-gcn. This might be valuable for more in depth discussion in our report.

Communication with group 18: Meeting was held and draft reports were traded.

- Alex: 15h - meetings/training/coding
- Sixten: 10h - 4h meetings, 6h writing and editing.
- Max: 8h - 2h meetings, 5h writing and editing, 1h testing trained models.
- Emilia: 12h - 4h meetings, 8h writing and editing.

L. Week 47

Summary: This week we had a meeting to prepare slides and presentation. Everyone in the group was then assigned different parts of the presentation to prepare and polish.

Next week:

Communication with group 18:

- Alex: 3h, 1h meeting, 2h presentation
- Sixten: 2h, 1h meet, 1h presentation
- Max: 3h 1h meetings, 2h presentation
- Emilia: 2h, 1h meetings, 1h presentation

M. Week 48

Summary: This week we held our final presentations for SEB, showing them our work and finalizing this project. The week was mostly spent on preparing for the presentation, practicing and improving upon the presentation slides and content.

Next week: Next week, we will look over the report a final time and fill in the parts where we want to improve as well as practice for the final report at KTH. We will aslo look over the opposing group's report in order to prepare some questions for them.

Communication with group 18: This week we held our presentations for SEB and were able to ask some questions regarding each others project.

- Alex: 4h: 3h meetings and 1h presentation.
- Sixten: 4h: 3h meetings and 1h presentation.
- Max: 7h: 5h research and presentation, 2h meetings
- Emilia: 7h: 4h making layout for presentation and working on presentation, 3h meetings

N. Week 49

Summary: We didn't do much this week except discuss some technical details regarding the presentation next week and an approximate timeline for how to finish off the course. We also all read through the opposing groups report and formulated some questions to ask.

Next week: Present at KTH. Sixten will will write future work. Revise report according to feedback given at presentation and then (hopefully) hand it in.

Communication with group 18: Very little this week. Some talk regarding report and presentation logistics.

- Alex: 4h - 2h, 1h revise presentation, 1h read opposing group report.
- Sixten: 4h - 2h, 1h revise presentation, 1h read opposing group report.
- Max: 4h - 2h, 1h revise presentation, 1h read opposing group report.
- Emilia: 4h - 2h, 1h revise presentation, 1h read opposing group report.

O. Week 50

Summary: Prepared our presentation for KTH and did the presentation. We also finalized the report as a group.

Next week:

Communication with group 18: We exchanged some questions during the KTH meeting.

- Alex: 4h
- Sixten: 3.5h
- Max: 4h: 1h presentation, 3 hours report
- Emilia: 4h: 1h presentation, 2h prepare for presentation and 1h finalize report